

jQuery: The Definitive Guide

jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers. With a combination of versatility and extensibility, jQuery has changed the way that millions of people write JavaScript.

Originally created by John Resig in 2006, jQuery's philosophy is "Write Less, Do More." Before jQuery, developers had to write complex code to handle browser inconsistencies (especially with Internet Explorer). jQuery provided a unified wrapper that smoothed out these differences.

Definition: jQuery is a DOM-manipulation library that wraps standard JavaScript to provide a cleaner syntax and cross-browser reliability.

The Evolution of jQuery

While modern browsers and "Vanilla JS" have adopted many features that jQuery popularized (like `querySelector`), jQuery remains relevant in legacy systems, rapid prototyping, and for its robust plugin ecosystem. It continues to be one of the most deployed JavaScript libraries in the world.

Why jQuery?

jQuery was designed to solve several major pain points in web development. Here are the primary reasons why it became the industry standard:

- **Write Less, Do More:** jQuery significantly reduces the lines of code required to perform common tasks. A 10-line JavaScript function might only take 1 or 2 lines in jQuery.
- **Cross-Browser Support:** One of jQuery's greatest strengths is its ability to handle cross-browser issues. It works consistently across Chrome, Firefox, Safari, Edge, and older versions of IE.
- **Simplified DOM Manipulation:** Selecting elements, changing content, and modifying attributes is intuitive using CSS-style selectors.
- **AJAX Simplified:** Handling asynchronous requests is notoriously difficult in native JS (pre-fetch API). jQuery provides simple methods like `$.ajax()` and `$.get()`.
- **Robust Community:** Thousands of plugins exist for jQuery, ranging from image sliders to date pickers and form validation.

```
// Vanilla JS Select

document.getElementById('myId').addEventListener('click',
function() { ... });

// jQuery Select
$('#myId').click(function() { ... });
```

Adding jQuery to Your Project

There are two main ways to include jQuery in your web project: using a Content Delivery Network (CDN) or downloading the library locally.

1. Using a CDN (Recommended)

A CDN is a network of servers that deliver the library based on the user's location. This often leads to faster load times because the user may already have a cached version from another site.

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

2. Local Installation

You can download the `.js` file from jquery.com and host it on your own server. This is useful for offline development.

```
<script src="js/jquery-3.6.0.min.js"></script>
```

Note: Always place your script tags just before the closing `</body>` tag to ensure the HTML elements are loaded before the script attempts to interact with them.

The jQuery Syntax

The jQuery syntax is tailor-made for selecting HTML elements and performing some action on the elements.

```
$(selector).action()
```

- **\$**: A sign to define/access jQuery.
- **(selector)**: To "query" (or find) HTML elements.
- **action()**: A jQuery action to be performed on the element(s).

The Most Used Selectors

Selector	Description
<code>\$("*")</code>	Selects all elements
<code>\$(this)</code>	Selects the current HTML element
<code>\$("p.intro")</code>	Selects all <p> elements with class="intro"
<code>\$("p:first")</code>	Selects the first <p> element
<code>\$("ul li:first")</code>	Selects the first of the first
<code>\$("[href]")</code>	Selects all elements with an href attribute

DOM Manipulation: Getting and Setting

jQuery provides powerful methods to change and manipulate HTML elements and attributes.

Content Manipulation

- `text()` - Sets or returns the text content of selected elements.
- `html()` - Sets or returns the content of selected elements (including HTML markup).
- `val()` - Sets or returns the value of form fields.

Code Example

```
$("#btn-text").click(function(){
    alert("Text: " + $("#test").text());
});
$("#btn-html").click(function(){
    alert("HTML: " + $("#test").html());
});
```

Attribute Access

The `attr()` method is used to get or set attribute values. For example, changing a link's destination:

```
$("#button").click(function(){
    $("#link").attr("href", "https://
www.codehive.com");
```

```
});
```

CODEHIVE

DOM Manipulation: Add and Remove

In this section, we cover how to add new content and remove existing elements from the DOM.

Adding New HTML Content

- `append()` - Inserts content at the end of the selected elements.
- `prepend()` - Inserts content at the beginning of the selected elements.
- `after()` - Inserts content after the selected elements.
- `before()` - Inserts content before the selected elements.

Removing Elements

- `remove()` - Removes the selected element (and its child elements).
- `empty()` - Removes the child elements from the selected element.

You can also filter the elements to be removed. For example, `$(
("p").remove(".test")` removes all `<p>` elements with `class="test"`.

Manipulating CSS Classes

Instead of setting individual style properties, it is often better practice to toggle predefined CSS classes.

- `addClass()` - Adds one or more classes to the selected elements.
- `removeClass()` - Removes one or more classes from the selected elements.
- `toggleClass()` - Toggles between adding/removing classes from the selected elements.
- `css()` - Sets or returns the style attribute.

The `css()` Method

To return the value of a specific CSS property, use: `css("propertyname")`.

To set a specific CSS property, use: `css("propertyname", "value")`.

```
// Setting multiple properties
$("p").css({
    "background-color": "yellow",
    "font-size": "200%"
});
```

jQuery Events

All the different users' actions that a web page can respond to are called events. jQuery is particularly good at managing these.

Common DOM Events

- **Mouse Events:** click, dblclick, mouseenter, mouseleave
- **Keyboard Events:** keypress, keydown, keyup
- **Form Events:** submit, change, focus, blur
- **Document/Window Events:** load, resize, scroll, unload

The on() Method

The `on()` method attaches one or more event handlers for the selected elements. It is the modern and preferred way to handle events in jQuery.

```
$( "p" ).on({  
    mouseenter: function(){  
        $(this).css("background-color",  
"lightgray");  
    },  
    mouseleave: function(){  
        $(this).css("background-color",  
"lightblue");  
    },  
    click: function(){  
        $(this).css("background-color",  
"yellow");  
    }  
});
```

| jQuery Effects

jQuery makes it very easy to add visual effects to your website.

Hiding and Showing

`hide()` and `show()` allow you to toggle the visibility of elements with a speed parameter (slow, fast, or milliseconds).

Fading

- `fadeIn()`
- `fadeOut()`
- `fadeToggle()`
- `fadeTo()` (allows fading to a specific opacity)

Sliding

- `slideDown()`
- `slideUp()`
- `slideToggle()`

Animate

The `animate()` method is used to create custom animations.

```
$("#button").click(function(){
    $("#div").animate({
        left: '250px',
        opacity: '0.5',
        height: '150px',
        width: '150px'
    });
});
```

```
});
```

CODEHIVE

| jQuery AJAX

AJAX is the art of exchanging data with a server, and updating parts of a web page - without reloading the whole page.

What is AJAX?

AJAX = Asynchronous JavaScript And XML. In short; AJAX is about loading data in the background and displaying it on the webpage, without reloading the whole page.

Methods

- `$.get(URL, callback)` : Requests data from the server with an HTTP GET request.
- `$.post(URL, data, callback)` : Requests data from the server using an HTTP POST request.
- `load(URL, data, callback)` : Loads data from a server and puts the returned HTML into the selected element.

The Comprehensive \$.ajax() method

```
$.ajax({  
    url: "api/data",  
    type: "GET",  
    success: function(result) {  
        console.log("Data received",  
result);  
    },  
    error: function(xhr, status, error) {  
        console.error("Error", error);  
    }  
});
```

Ready, Set, Go!

You might have noticed that almost all jQuery methods in our examples are inside a document ready event:

```
$(document).ready(function(){  
    // jQuery methods go here...  
});
```

Why is this important? This is to prevent any jQuery code from running before the document is finished loading (is ready). It is good practice to wait for the document to be fully loaded and ready before working with it. This also allows you to have your JavaScript code before the body of your document, in the head section.

Here are some examples of actions that can fail if methods are run before the document is fully loaded:

- Trying to hide an element that is not created yet.
- Trying to get the size of an image that is not loaded.

The Shorthand

The jQuery team has also created an even shorter method for the document ready event:

```
$(function(){  
    // jQuery methods go here...  
});
```

★ Exam Summary Points

- **Lightweight:** jQuery is a "Write Less, Do More" JavaScript library.
- **Cross-browser:** It handles cross-browser compatibility issues automatically.
- **Syntax:** Uses the `$` symbol for selection. `$(selector).action()`
- **DOM:** Simplifies DOM selection, traversal, and manipulation.
- **AJAX:** Makes asynchronous server communication easy for all browsers.
- **Ready:** Always use `$(document).ready()` to ensure the DOM is safe to manipulate.
- **Chaining:** You can chain actions. Example: `$("#p1").css("color", "red").slideUp(2000).slideDown(2000);`

Best Practices

1. **Use CDNs:** Use public CDNs for faster loading.
2. **Minimize DOM access:** Selecting elements is expensive. If you use a selector more than once, store it in a variable (e.g., `const $btn = $("#btn");`).
3. **Chain your actions:** Use chaining to improve performance and code readability.
4. **Keep it simple:** If you only need one or two simple functions, evaluate if vanilla JavaScript might be more efficient for your specific use case.